IFB104 Practice Quiz: Building IT Systems

Question 1

Alice is helping her tutor write a program that converts a student's numeric score into a grade. Before handing it over, she tests it with a score of 90. Here is Alice's code:

```
score = 90

if score >= 50:
    grade = 'Pass'
if score >= 70:
    grade = 'Credit'
elif score >= 85:
    grade = 'Distinction'
else:
    grade = 'Fail'

print(grade)
```

What is printed when this code runs?

A. `Pass`

B. `Distinction`

C. `Credit`

D. `Fail`

E. `Pass` then `Credit` on separate lines

Question 2 (multiple answer)

Bob is building a small weather app that suggests what to do for the day, depending on the temperature and whether it is raining. He writes the following function:

```
def choose_activity(temperature, raining):
    if temperature > 30 or raining and temperature > 20:
        return 'Swim'
    elif not raining:
        return 'Walk'
    else:
        return 'Read'
```

Which of the following function calls return `'Swim'`? Select all that apply.

A. `choose_activity(25, False)`

B. `choose_activity(35, False)`

C. `choose_activity(15, True)`

D. `choose_activity(25, True)`

E. `choose_activity(20, True)`

Question 3

Charlie is a tutor who has decided to give every student a 10-mark bonus. He writes a small helper function and tests it on a mark of 50:

```
def add_bonus(score):
    score = score + 10

mark = add_bonus(50)
print(mark)
```

What is printed when this code runs?

A. `60`

B. `50`

C. `None`

D. `10`

E. `TypeError`

Question 4

Priya is writing a shopping-list app. She writes one function to add an item to the basket, and another to reset the basket by replacing it with a placeholder. She then tests both functions:

```
def add_item(basket, item):
    basket.append(item)

def replace_basket(basket):
    basket = ['empty']

shopping = ['milk']
add_item(shopping, 'eggs')
replace_basket(shopping)
print(shopping)
```

What is printed when this code runs?

A. `['milk']`

B. `['empty']`

C. `['empty', 'eggs']`

D. `['milk', 'eggs']`

E. `['eggs']`

Question 5

Minh wants to find out whether a function can change a variable that is passed to it. To make the experiment as clear as possible, he gives the variable and the function's parameter the same name, `a`:

```
def modify(a):
    a = a + 5

a = 5
modify(a)
print(a)
```

What is printed when this code runs?

A. `10`

B. `5`

C. `None`

D. `UnboundLocalError`

E. `NameError`

Question 6

Sofia is building a sign-up form for a gym, and every new password must contain at least one digit. She writes the following function to check this rule, and it seems to work when she tests it with `'1abc'`:

```
def check_password(password):
    for character in password:
        if character.isdigit():
            return True
        else:
            return False
```

What does this function actually check?

A. Whether the password contains at least one digit

B. Whether every character in the password is a digit

C. Whether the first character of the password is a digit

D. Whether the last character of the password is a digit

E. How many digits the password contains

Question 7

Alice wants to print every number in her list on a single line. Instead of looping over the list directly, she decides to loop over the index positions and use each position to look up a number:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]

for position in range(1, len(numbers) + 1):
    print(numbers[position], end=' ')
```

What is the result of running this code?

A. `1 2 3 4 5 6 7 8 9` is printed and the program ends normally

B. `1 2 3 4 5 6 7 8 9` is printed, then an `IndexError` is raised

C. `2 3 4 5 6 7 8 9` is printed and the program ends normally

D. An `IndexError` is raised before anything is printed

E. `2 3 4 5 6 7 8 9` is printed, then an `IndexError` is raised

Question 8

Bob is writing a maths helper for his younger sister. A divisor of a whole number divides into it exactly, with no remainder; for example, the divisors of 6 are 1, 2, 3 and 6. Bob reasons that `number % candidate` is zero exactly when `candidate` divides `number`, so he writes the following function to count the divisors and tests it with 6:

```
def count_divisors(number):
    count = 0
    for candidate in range(number + 1):
        if number % candidate == 0:
            count = count + 1
    return count

print(count_divisors(6))
```

What is the result of running this code?

A. Nothing is printed; a `ZeroDivisionError` is raised

B. `4`

C. `3`

D. `5`

E. `0`

Question 9

Priya is organising a pizza night for her study group. There are 19 slices to share between 5 friends, and she wants to know how many whole slices each friend gets and how many are left over. Out of curiosity, she also tries the same operator with a negative number:

```
slices = 19
friends = 5

print(slices // friends)
print(slices % friends)
print(-17 // 5)
```

What is printed when this code runs?

A. `3`, `4` and `-3`

B. `4`, `4` and `-3`

C. `3.8`, `4` and `-3.4`

D. `3`, `4` and `-4`

E. `4`, `1` and `-4`

Question 10

Charlie is marking a first-year student's work and finds four lines of code that each crash the program. He runs each line on its own to see which exception it raises:

```
'3' + 4                 # Line 1
{'apple': 3}['banana']  # Line 2
[10, 20][2]             # Line 3
int('four')             # Line 4
```

Which exception does each line raise, in order from Line 1 to Line 4?

A. `ValueError`, `IndexError`, `KeyError`, `TypeError`

B. `TypeError`, `IndexError`, `KeyError`, `ValueError`

C. `SyntaxError`, `KeyError`, `IndexError`, `TypeError`

D. `TypeError`, `KeyError`, `IndexError`, `ValueError`

E. `TypeError`, `KeyError`, `IndexError`, `TypeError`

Question 11

Priya is a tutor who keeps her class's results in a list of lists, just like the data in Assignment 1. A student passes only if they score at least 50 in both the exam and the assignment. She wants the code below to print `['Ana', 'Dee']`, but three parts of it are missing:

```
results = [['Ana', 72, 85],
           ['Ben', 45, 38],
           ['Cy', 90, 49],
           ['Dee', 50, 50]]
NAME, EXAM, ASSIGNMENT = 0, 1, 2

passed = []
for student in results:
    if ___(1)___ >= 50 and ___(2)___ >= 50:
        passed.append(___(3)___)

print(passed)
```

How should the three blanks be filled in? Select the code snippets in the order the blanks appear.

A. `EXAM`, `ASSIGNMENT`, `NAME`

B. `student[EXAM]`, `student[ASSIGNMENT]`, `student[NAME]`

C. `results[EXAM]`, `results[ASSIGNMENT]`, `results[NAME]`

D. `student[1]`, `student[2]`, `student`

E. `student[NAME]`, `student[EXAM]`, `student[ASSIGNMENT]`

Question 12

Minh writes a function to add up the prices in a shopping cart, but makes a small typing mistake on the last line of the function without noticing. He then runs his program:

```
def total_price(prices):
    total = 0
    for price in prices:
        total = total + price
    return totl

print('Start')
result = total_price([2, 3])
print(result)
```

What happens when this program runs?

A. A `SyntaxError` is reported and nothing is printed

B. A `NameError` is raised before `Start` is printed

C. `Start` is printed, then a `NameError` is raised

D. `Start` is printed, then `5`

E. `Start` is printed, then `None`

Question 13

Sofia is building a sign-up window for her local gym using PySide6. The window needs:

1. a box to type the user's name (one line),
2. an on/off option to subscribe to the newsletter,
3. a choice of exactly one plan (Basic, Pro or Team), with all three options visible at once,
4. a button to submit the form.

Which widgets best match items 1 to 4, in order?

A. `QTextEdit`, `QRadioButton`, `QComboBox`, `QLabel`

B. `QLineEdit`, `QRadioButton`, `QCheckBox`, `QPushButton`

C. `QLabel`, `QCheckBox`, `QListWidget`, `QPushButton`

D. `QLineEdit`, `QCheckBox`, `QRadioButton`, `QPushButton`

E. `QLineEdit`, `QPushButton`, `QComboBox`, `QLabel`

Question 14

Alice is building a suburb search tool for a real estate agency. She writes the following code to lay out the main window, where `window` is a `QWidget`:

```
main_layout = QVBoxLayout()

search_row = QHBoxLayout()
search_row.addWidget(QLabel('Suburb:'))
search_row.addWidget(QComboBox())

main_layout.addLayout(search_row)
main_layout.addWidget(QPushButton('Search'))
main_layout.addWidget(QLabel('Results'))

window.setLayout(main_layout)
```

Which sketch best shows the resulting window?

A.

```
+-----------------------------------+
| Suburb:  [ combo box        v ]   |
| [            Search            ]  |
| Results                           |
+-----------------------------------+
```

B.

```
+-----------------------------------+
| Suburb: [combo v] [Search] Results |
+-----------------------------------+
```

C.

```
+-----------------------------------+
| Suburb:                           |
| [ combo box                 v ]   |
| [            Search            ]  |
| Results                           |
+-----------------------------------+
```

D.

```
+-----------------------------------+
| [            Search            ]  |
| Results                           |
| Suburb:  [ combo box        v ]   |
+-----------------------------------+
```

E.

```
+-----------------------------------+
| Suburb:         | [ Search ]      |
| [ combo   v ]   | Results         |
+-----------------------------------+
```

Question 15

Bob adds a button to his first Qt program and wants a message to be printed every time the button is clicked. He writes:

```
def show_message():
    print('Button clicked!')

button = QPushButton('Click me')
button.clicked.connect(show_message())
```

What happens when the last line runs?

A. Nothing happens yet; `Button clicked!` is printed each time the user clicks the button

B. `Button clicked!` is printed once straight away, then `connect` raises a `TypeError` because it was given `None`

C. Nothing is ever printed, but no error occurs

D. A `SyntaxError` is reported because `connect` needs a string

E. `Button clicked!` is printed twice on every click

Question 16

Priya is writing a script to pick out QUT student IDs from a long file of emails. A QUT student ID is the lowercase letter `n` followed by exactly eight digits, for example `n12345678`, and must appear as a complete token, not embedded in other characters. Her pattern has two parts missing:

```
import re

student_ids = re.findall(r"___(1)___n\d___(2)___\b", email_text)
```

How should the two blanks be filled in?

A. `\b` and `{8}`

B. `^` and `+`

C. `\b` and `{1,8}`

D. `\s` and `{8}`

E. `\B` and `*`

Question 17

Minh keeps short notes of what he spends each day, such as `"Coffee 4, lunch 12, bus 3"`. He writes the following function and runs it on each note:

```
import re

def mystery(note):
    total = 0
    for number in re.findall(r"\d+", note):
        total = total + int(number)
    return total
```

What is the purpose of this function?

A. It counts how many digits appear in the note

B. It adds up every single digit in the note, so `12` counts as `1 + 2`

C. It always raises a `TypeError`, because `findall` returns strings

D. It adds up all the whole numbers in the note, so the example gives `19`

E. It returns the largest number in the note

Question 18 (multiple answer)

Sofia manages a warehouse where every product code must be exactly two uppercase letters followed by exactly three digits. She checks each code that staff type in using `re.fullmatch(pattern, text)` with the pattern below:

```
import re

pattern = r"[A-Z]{2}\d{3}"
```

For which of the following values of `text` does `re.fullmatch` return a match? Select all that apply.

A. `'AB123'`

B. `'ab123'`

C. `'AB1234'`

D. `'XY000'`

E. `'A B123'`

Question 19* (multiple answer)

Alice's lecturer sets the class a challenge: print the numbers 1 to 5 in ascending order, one per line, **without using any loop**. Alice and her classmates submit the five programs below, all using a recursive function. Which of them meet the challenge? Select all that apply.

A.

```
def count_up(number):
    if number > 5:
        return
    print(number)
    count_up(number + 1)

count_up(1)
```

B.

```
def count_up(number):
    if number > 5:
        return
    count_up(number + 1)
    print(number)

count_up(1)
```

C.

```
def count_up(number):
    print(number)
    count_up(number + 1)

count_up(1)
```

D.

```
def count_up(number):
    if number == 0:
        return
    count_up(number - 1)
    print(number)

count_up(5)
```

E.

```
def count_up(number):
    if number < 5:
        print(number)
        count_up(number + 1)

count_up(1)
```

Question 20

Before submitting his assignment, Bob's tutor asks him to tidy up his code so that it follows the IFB104 Code Presentation Guide. His classmates each suggest a different way of converting minutes into seconds. Which suggestion best follows the guide?

A.

```
for i in range(len(minute_list)):
    second_list.append(minute_list[i] * 60)
```

B.

```
SECONDS_PER_MINUTE = 60
total_seconds = minutes * SECONDS_PER_MINUTE
```

C.

```
x = m*60 # multiply m by 60
```

D.

```
def Calc(m): return m*60
```

E.

```
totSecs = mins * 60 # calc secs
```

Question 21*

Charlie is studying the Fibonacci sequence `0, 1, 1, 2, 3, 5, 8, 13, ...`, in which each term is the sum of the two terms before it. He stores each term in a list so that every new term can be built from the two before it. His function is meant to return term `n`, counting from 0, so `fibonacci(6)` should return `8`:

```
def fibonacci(n):
    fibo = [0] * n
    fibo[0] = 0
    fibo[1] = 1
    for index in range(2, n + 1):
        fibo[index] = fibo[index - 1] + fibo[index - 2]
    return fibo[n]

print(fibonacci(6))
```

What happens when this code runs, and what is the best fix?

A. It prints `8`; no fix is needed

B. It prints `5`; change the return statement to `return fibo[n + 1]`

C. It raises an `IndexError`; change the first line to `fibo = [0] * (n + 1)`

D. It raises an `IndexError`; change the loop to `range(2, n)` and leave everything else as it is

E. It loops forever; add a `break` inside the loop
